SIMATS - SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 3

CO3: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Design, Implement and Validate Constructors, Destructors and Operator Overloading Concepts in C++ Programs

Student Name:	Aadarsh. B
Register Number:	192565009
Course Code:	DSA0102
Compiler:	g++ 13
GitHub Repository:	https://github.com/aadarshb2565/DSA01-192565009-Assignment3

Submission Date: 13.11.2026

DECLARATION

I hereby declare that this assignment is my own individual work. I have executed every task including all "Complete the Code" programs. The GitHub repository contains my progressive commits (at least one per task). All code compiles and runs correctly on the stated compiler.

Signature: _____

Date: _____

TABLE OF CONTENTS

1. Task 1: Constructors and Constructor Overloading

2. Task 2: Copy Constructors: Shallow and Deep Copies

3. Task 3: Dynamic Constructors and Destructors

4. Task 4: Overloading Arithmetic and Stream Operators

5. Task 5: Overloading Increment, Comparison and Subscript Operators

6. Task 6: The Rules of Operator Overloading

7. Task 7: Design a Complex-Number Class from First Principles

TASK 1: CONSTRUCTORS AND CONSTRUCTOR OVERLOADING

Task 01 - Call by Value, Reference, and Pointer

Predict the Output

Program:

My predicted output:

Explanation:

- change(a, b, &c): a is passed by value (local copy, no change to original), b is passed by reference (original changes), c is passed by pointer (original changes).

- bigger(a, c) = 100: bigger returns a reference to the larger of a (1) and c (13), which is c. Assigning 100 to the result changes c to 100.

- show(1): uses defaults y=5, z=10. Sum = 1+5+10 = 16.

- show(1, 2): uses default z=10. Sum = 1+2+10 = 13. Wait, let me re-calculate. show(1, 2) is 1+2+10 = 13.

- show(1, 2, 3): Sum = 1+2+3 = 6.

Revised Prediction:

Implementations

1. Concession Function (Three Ways)

Task01/concession.cpp implements the three ways:

- By Value: void apply(double fee, ...)

- By Reference: void apply(double& fee, ...)

- By Pointer: void apply(double* fee, ...)

2. Complete the Code

Task01/complete_code.cpp implements the required TODO.

Inquiry & Design

Parameter Storage:

- Call by Value: Parameter is stored in a new memory location on the stack (local copy). Changes are local.

- Call by Reference: Parameter is an alias for the original variable. No new memory for the value is allocated; it points to the original.

- Call by Pointer: Parameter is a local variable that stores the address of the original variable.

Const Reference: Use const T& when passing large objects (like a Student class) to avoid expensive copies while ensuring the function cannot modify the original object.

Design Decision:

For multiple concessions, I recommend returning the new fee or using call by reference. However, returning the value is often cleaner for chaining (e.g., fee = apply(apply(fee, 10), 5)), whereas in-place modification is more memory efficient for very large objects. For a simple long long fee, returning the value is preferred for functional purity.

Alternative Concept and Complexity Analysis

Returning Value vs Reference:

If long long applyConcession(long long fee, int percent) is used, the caller must assign the result back: hostelFee = applyConcession(hostelFee, 10);.

Space Complexity Analysis:

- long long (8 bytes):

- By Value: 8 bytes copied.

- By Reference: 8 bytes (pointer size) copied.

- Negligible difference.

- Large Student Object (e.g., 1 KB):

- By Value: 1024 bytes copied every call.

- By Reference/Const Reference: 8 bytes copied.

- Significant performance gain with references.

Time Complexity: All versions are $O(1)$.

TASK 2: COPY CONSTRUCTORS: SHALLOW AND DEEP COPIES

Task 02 - Return by Reference and Its Dangers

Implementation

1. High Mark Moderation

Task02/marks_register.cpp implements:

- int& findHighest(int arr[], int size): Returns a reference to the highest element.

- Moderation logic: findHighest(marks, n) += 2; (using the call on the left of =).

2. The Dangling Reference Danger

Task02/dangling_ref.cpp demonstrates returning a reference to a local variable.

- Compiler Warning: Modern compilers (like g++) issue a warning: warning: reference to local variable 'max' returned.

- Undefined Behaviour: The local variable is destroyed when the function exits. Returning a reference to it means the caller gets a "dangling reference" to a memory location that may be overwritten by other functions, leading to unpredictable values or segmentation faults.

3. Reference vs Const Reference vs Copy

- Return by Reference (int&): Returns an alias. Allows modification of the original. Fast (no copy).

- Return by Const Reference (const int&): Returns an alias but prevents modification. Fast.

- Return by Copy (int): Returns a new value. Original cannot be modified. Slightly slower for large objects, but safest.

Changed Condition: std::vector Growth

When using a std::vector, if a reference to an element is obtained (e.g., int& ref = vec[0];) and the vector subsequently grows (via push_back), the vector may reallocate its internal

storage to a new, larger memory block. The old memory is freed, making the previously obtained reference invalid (dangling).

Alternative Concept and Complexity Analysis

Index vs Pointer vs Reference

- Index (int): int idx = findHighestIndex(marks, n); marks[idx] += 2;

- Pointer (int): int ptr = findHighestPtr(marks, n); *ptr += 2;

- Reference (int&): findHighest(marks, n) += 2;

Complexity Analysis:

- Time Complexity: All three are $O(n)$ as they must scan the array once.

- Space Complexity: All three are $O(1)$.

Safety Comparison (with Growing Vector):

- Reference/Pointer: Extremely unsafe. Both become invalid immediately upon reallocation.

- Index: Safest. The index remains valid as long as the element is not deleted, regardless of where the vector moves in memory.

Evidence Expected

- marks_register.cpp: Working moderation via reference.

- dangling_ref.cpp: Code causing compiler warnings and exhibiting UB.

- Analysis of vector reallocation risks.

TASK 3: DYNAMIC CONSTRUCTORS AND DESTRUCTORS

Task 03 - Default Arguments and Function Overloading

The Ambiguity Problem

Scenario:

The developer implemented:

1. charge(double weight)

2. charge(double weight, bool express)

3. charge(double weight, bool express = false, bool insured = false)

Why it fails to compile:

The call charge(10.5) is ambiguous. The compiler cannot decide whether to call function (1) with one argument or function (3) using its default arguments for express and insured. Similarly, charge(10.5, true) matches both (2) and (3).

Redesigned Solution

To resolve ambiguity and handle different input types, I have used Function Overloading based on types and a single function with Default Arguments for the options.

Overloaded Signatures:

1. double charge(int weightGrams) $\rightarrow$ Handles weight in grams.

2. double charge(double weightKg) $\rightarrow$ Handles weight in kilograms.

3. double charge(const Parcel& p) $\rightarrow$ Handles a Parcel object.

4. double charge(double weightKg, bool express, bool insured) $\rightarrow$ The base calculation logic.

To keep common calls short, I provide a wrapper or use the most general overload with defaults:

double charge(double weightKg, bool express = false, bool insured = false)

Predicted Selections:

| Call | Selected Function | Reason |

|---|---|---|

| charge(500) | charge(int) | Exact match for int. |

| charge(2.5) | charge(double, bool=false, bool=false) | Exact match for double. |

| charge(2.0f) | charge(double, ...) | float promotes to double. |

| charge(2.5, true) | charge(double, bool, bool=false) | Exact match for double, bool. |

| charge(myParcel) | charge(const Parcel&) | Exact match for Parcel object. |

| charge(2.5, true, true) | charge(double, bool, bool) | Exact match for all three. |

Inquiry

Overload Resolution Process:

1. Exact Match: Compiler looks for a function where types match exactly.

2. Promotion: If no exact match, types are promoted (e.g., char $\rightarrow$ int, float $ \rightarrow$ double).

3. Standard Conversion: If still no match, it tries implicit conversions (e.g., int $ \rightarrow$ double).

4. Ambiguity: If multiple functions match equally well, the compiler throws an error.

Right-to-Left Defaults:

Default arguments must be given from right to left because the compiler assigns arguments positionally. If a middle argument had a default but the rightmost didn't, the compiler wouldn't know if a missing argument was intended for the middle one or if the rightmost one was simply omitted.

Alternative Concept and Complexity Analysis

Alternative: Single Parcel Structure

Instead of overloading, we could use a ChargeRequest struct:

Comparison:

- Overload Resolution Time: Happens at compile time. There is zero runtime cost for deciding which function to call.

- Space Complexity:

- Overloading: Small stack frames for each call.

- Struct: Passes one reference (8 bytes) to the function.

- Extensibility: Adding a new option (e.g., fragile) requires changing the function signature in overloading (breaking existing calls if not defaulted), whereas adding a field to the ChargeRequest struct preserves the function signature.

TASK 4: OVERLOADING ARITHMETIC AND STREAM OPERATORS

Task 04 - Specifying a Class with an Array Inside It

Implementation

1. SavingsAccount Class

Task04/savings_account.cpp implements:

- Private Data:

- std::string accountNumber

- double balance

- double transactions[10] (Fixed array for mini-statement)

- int count (Number of transactions)

- int head (Circular index for the most recent transaction)

- Public Member Functions:

- deposit(double amount): Adds to balance, records transaction.

- withdraw(double amount): Subtracts from balance (if sufficient), records transaction.

- miniStatement(): Prints the last 10 transactions in chronological order.

2. Member Function Definition

- Inside Class: deposit() and withdraw() are defined inside the class for brevity.

- Outside Class: miniStatement() is defined outside the class using the scope resolution operator SavingsAccount::miniStatement().

Inquiry & Design

Why the array must be private:

The transaction array and the head index must be private to prevent external code from arbitrarily modifying the transaction history or corrupting the circular index, which would lead to incorrect mini-statements.

Handling the 11th Transaction:

- Shifting the Array: Every time a new transaction arrives after the 10th, we shift elements 1-9 to 0-8.

- Cost: $O(k)$ where k is array size.

- Circular Index (Implemented): We use a head index that wraps around using the modulo operator: head = (head + 1) % 10.

- Cost: $O(1)$.

- Logic: The newest element always overwrites the oldest.

Design Decision (Expanding to 50 Transactions):

Increasing the capacity to 50 is a trivial change if using a fixed array (just change the constant 10 to 50). The circular index design is highly scalable as it maintains $O(1)$ insertion regardless of array size.

Alternative Concept and Complexity Analysis

Alternative: std::deque vs Circular Buffer

- std::deque: Use push_back() and if size() > 10, pop_front().

- Time (Add): $O(1)$.

- Space: Dynamic allocation overhead.

- Circular Buffer (Fixed Array): Use a fixed array and a wrap-around index.

- Time (Add): $O(1)$.

- Space: Minimal (exactly 10 * sizeof(double)).

Comparison:

For a small, fixed number of transactions (10 or 50), the Circular Buffer with a fixed array is superior because it avoids dynamic memory allocation and fragmentation while maintaining optimal time complexity.

TASK 5: OVERLOADING INCREMENT, COMPARISON AND SUBSCRIPT OPERATORS

Task 05 - Static Members and Arrays of Objects

Implementation

1. BusTicket Class Design

Task05/bus_tickets.cpp implements the ticket system:

- Static Data Member: static int issued; (Shared across all objects to track total seats).

- Static Member Function: static int count(); (Returns the total issued tickets without needing a specific object).

- Logic: The constructor increments issued and assigns a seat number only if issued <= CAPACITY. Otherwise, the ticket is marked as "Waiting".

2. Array of Objects

The program creates an array BusTicket trip[4] (where capacity is 3), demonstrating that the 4th object correctly enters the waiting list.

Inquiry & Design

Static Member Placement:

- Declaration: Inside the class.

- Definition: Outside the class (e.g., int BusTicket::issued = 0;). This is mandatory because the static member exists independently of any object; the definition allocates the actual memory for it.

Static Member Functions:

A static member function cannot access non-static members because it doesn't have a this pointer. It doesn't belong to any specific instance, so it doesn't know which object's data to access. It can only access other static members.

Behavior in Loops/Functions:

Since issued is static, its value persists across all function calls and loop iterations. Even if BusTicket objects are created inside a local function that returns, the issued count remains incremented for the next set of objects created elsewhere.

Predict the Output

Program Snippet:

My predicted output:

Explanation:

- Token counter[3] runs the constructor 3 times: next becomes 101, 102, 103.

- Token extra runs the constructor 1 time: next becomes 104.

- counter[0].get() is 101.

- counter[2].get() is 103.

- extra.get() is 104.

- Token::issued() is 104 - 100 = 4.

- Total constructor calls: 4.

Alternative Concept and Complexity Analysis

Alternative: Global Variable vs TicketOffice Object

- Global Variable: int totalIssued;

- Pros: Simple.

- Cons: No encapsulation; any function can reset the count.

- TicketOffice Object: TicketOffice office; passed to every BusTicket.

- Pros: Encapsulated. Allows multiple buses (each bus has its own TicketOffice).

- Cons: Higher boilerplate (passing pointers/references).

Complexity Analysis:

- Time Complexity: All $O(1)$ for incrementing and retrieving.

- Space Complexity: All $O(1)$ (one integer).

Verdict: Static members are best for single-instance shared state. For multiple buses, a TicketOffice object (or making BusTicket a member of a Bus class) is necessary.

TASK 6: THE RULES OF OPERATOR OVERLOADING

Task 06 - Friend Functions and Inline Functions

Implementation

1. Time Class Design

Task06/train_planner.cpp implements:

- Private Data: int h, m;

- Friend Functions: minutesBetween() and earlier().

- Inline Suggestion: Small functions like earlier are candidates for inline to reduce function call overhead.

Inquiry & Design

Why a Friend Function?

The minutesBetween and earlier functions operate on two Time objects equally. If implemented as a member function, one object would be the "owner" (this) and the other would be a parameter. A friend function provides a symmetric interface: earlier(t1, t2).

Friend vs Member Functions:

- Access: Both can access private members.

- Calling Convention: Member functions are called via t1.earlier(t2). Friend functions are called as standalone functions earlier(t1, t2).

- Context: Member functions have a this pointer; friend functions do not.

Inline Functions:

The inline keyword is a hint to the compiler to replace the function call with the actual function body. This eliminates the overhead of pushing arguments to the stack and jumping to a different address.

- When to use: Small, frequently called functions (like time comparisons).

Public Members Risk:

If h and m were public, any part of the program could set m = 99 or h = 25, breaking the internal consistency of the Time class. Encapsulation ensures that only valid times are stored.

Growth Investigation (Inline Performance)

Observation:

- -O0 (No Optimization): inline may provide a slight speedup by reducing call overhead.

- -O2 (Optimized): The compiler's optimizer often inlines functions automatically, regardless of the inline keyword. The performance difference between inline and non-inline usually disappears at high optimization levels.

Alternative Concept and Complexity Analysis

Alternative: Member Functions vs Getters

- Member Function (t1.earlier(t2)): Similar to friend functions. Encapsulated.

- Public Getters (t1.getHours()): Allows any function to compare times, but exposes internal state.

Comparison:

- Time Complexity: All $O(1)$.

- Encapsulation: Friend functions > Member functions > Getters. Friend functions keep data private while allowing specific, trusted external functions to access it.

Verdict: Friend functions are the best design here as they maintain strict encapsulation while providing a symmetric, clean API for comparisons.

TASK 7: DESIGN A COMPLEX-NUMBER CLASS FROM FIRST PRINCIPLES

Task 07 - Diagnose, Redesign and Validate

A. Diagnose the Limitation

Selected Program: Task04/savings_account.cpp (Mini-Statement)

Limitation: The current design uses a fixed-size array double transactions[10].

- Evidence: If the bank requirement changes to the last 50 or 100 transactions, we must manually change the array size and recompile.

- Core Issue: Fixed-size arrays are allocated on the stack and their size must be known at compile-time. They lack the flexibility to grow based on user needs or configuration.

B. Define the Design Objective

Objective: Implement a transaction history system that supports a configurable number of recent transactions (set at runtime) while maintaining $O(1)$ insertion time.

C. Design the Revised Solution

Redesign: Replace the fixed array with a std::vector<double> used as a circular buffer.

Changes:

1. Dynamic Size: The constructor now takes int maxHistory.

2. Vector Buffer: std::vector<double> transactions initialized to maxHistory.

3. Circular Logic: Retain the head index and modulo operator: head = (head + 1) % transactions.size().

Trade-off:

- Memory: Small overhead for std::vector metadata and heap allocation.

- Complexity: Transition from stack allocation to heap allocation.

D. Predict Before Implementing

Prediction: The redesigned system will handle any history size (e.g., 10, 50, 1000) without requiring code changes or recompilation, while keeping the deposit() and withdraw() operations at $O(1)$.

E. Implement and Validate

Task07/redesigned_savings.cpp implements this change.

Comparison:

| Feature | Original (Fixed Array) | Redesigned (Vector Circular) |

|---|---|---|

| Size Change | Requires Recompile | Runtime Configurable |

| Insertion Time | $O(1)$ | $O(1)$ |

| Memory Location | Stack | Heap |

| Scalability | Poor | Excellent |

F. Defend the Final Design

The objective was achieved. The system is now future-proof; if the bank decides to store 100 transactions, we simply pass 100 to the constructor. The original design was preferable only in extremely memory-constrained embedded systems where heap allocation is forbidden. For a banking application, the flexibility of std::vector far outweighs the negligible memory cost.